

Ejercicio Técnico Full Stack con IA Asistida

Objetivo

El objetivo de este ejercicio es evaluar la capacidad del candidato para utilizar herramientas de Inteligencia Artificial como apoyo en el desarrollo de software, analizar código generado automáticamente, identificar oportunidades de mejora y realizar modificaciones funcionales sobre una solución existente.

Durante la evaluación se prestará especial atención a:

- Proceso de trabajo y razonamiento técnico.
- Calidad de la solución entregada.
- Capacidad de aprendizaje.
- Capacidad de resolver problemas.
- Comprensión de código generado por terceros o por IA.
- Capacidad de comunicación técnica.
- Capacidad de identificar mejoras y oportunidades de optimización.

Premisa

Se desea desarrollar una aplicación de foro donde los usuarios puedan:

- Crear mensajes principales.
- Responder mensajes.
- Visualizar conversaciones en formato jerárquico.
- Permitir respuestas anidadas en múltiples niveles.

Como referencia funcional se puede utilizar:

<http://leonidasesteban.github.io/react-discussions/>

Tecnologías

Front-end: Angular

Back-end: Java

Persistencia: Archivo JSON

Control de versiones: GitHub

Herramientas de IA: Se permite y fomenta el uso de herramientas de Inteligencia Artificial como:

- GitHub Copilot
- ChatGPT
- Claude
- Gemini
- Cursor
- Otras similares

Actividades del Ejercicio

Parte 1: Generación de la solución mediante IA

Utilizando herramientas de IA, generar una primera versión funcional de la aplicación que cumpla con los requisitos de la premisa.

El candidato deberá documentar:

- Qué herramienta(s) utilizó.
- Qué prompts principales utilizó.
- Cómo fue refinando los resultados obtenidos.
- Qué decisiones tomó para adaptar o corregir el código generado.

Importante: No se evaluará quién escribe más código manualmente, sino cómo utiliza la IA para obtener resultados de calidad.

Parte 2: Análisis del código generado

Una vez obtenida la solución, el candidato deberá:

Explicar la arquitectura propuesta

Describir brevemente:

- Componentes principales.
- Flujo Front-end ↔ Back-end.
- Manejo de datos.
- Estrategia de persistencia.

Explicar el funcionamiento del código

Seleccionar las partes más relevantes y explicar:

- Cómo se crean los comentarios.
 - Cómo se almacenan.
 - Cómo se renderizan los niveles de respuestas.
 - Cómo se manejan las relaciones padre-hijo entre comentarios.
-

Parte 3: Identificación de mejoras

Analizar el código generado e identificar al menos **dos oportunidades de mejora**.

Para cada mejora deberá indicar:

- Problema identificado.
- Riesgo o impacto actual.
- Solución propuesta.
- Beneficio esperado.

Ejemplos:

- Rendimiento.
- Escalabilidad.
- Mantenibilidad.
- Seguridad.
- Diseño de componentes.
- Manejo de estado.
- Validaciones.
- Pruebas automatizadas.
- Tipado.

- Estructura de datos.
-

Parte 4: Cambio Funcional

La primera versión debe soportar un nivel de anidación definido por la IA o por el diseño original.

Posteriormente, el candidato deberá modificar la solución para que el número máximo de niveles de respuesta sea configurable.

Ejemplo

Inicialmente:

- Comentario
 - Respuesta Nivel 1
 - Respuesta Nivel 2

Posteriormente deberá poder configurarse para admitir:

- 3 niveles
- 5 niveles
- Ilimitados

El candidato deberá explicar:

- Qué partes del código modificó.
- Por qué realizó esos cambios.
- Qué implicaciones tienen esos cambios en el sistema.

Entregables

Repositorio GitHub

Debe incluir:

- Código fuente completo.
- Historial de commits.
- Instrucciones de ejecución.

Documento README

Debe incluir:

1. Uso de IA

- Herramientas utilizadas.
- Prompts más relevantes.
- Aprendizajes obtenidos.

2. Explicación de la solución

- Arquitectura.
- Componentes.
- Flujo de información.

3. Mejoras identificadas

- Al menos dos mejoras justificadas.

4. Cambio funcional realizado

- Explicación de la modificación para soportar diferentes niveles de respuestas.

5. Retos encontrados

- Problemas enfrentados.
- Cómo fueron resueltos.

Fecha: 01 / 09 / 2026

DD MM AAAA

Criterios de Evaluación

Criterio	Peso
Comprensión del código generado	30%
Calidad de la solución	20%
Capacidad de análisis y mejora	20%
Implementación del cambio funcional	20%
Comunicación y documentación	10%

Consideraciones Finales

- Se debe usar IA para el ejercicio, sin embargo, el candidato debe ser capaz de explicar el código y modificarlo
- Durante la revisión técnica se realizarán preguntas sobre el código entregado.
- El candidato debe ser capaz de justificar las decisiones tomadas y explicar el funcionamiento de la solución.

Este ejercicio busca evaluar la capacidad de trabajar en un entorno moderno de desarrollo asistido por IA, donde es tan importante generar código como comprenderlo, cuestionarlo y evolucionarlo.